

Nuno — Real-Time Multiplayer Card Game

A technical walkthrough: architecture, stack, and how each feature was built.

Backend: Node.js · TypeScript · Socket.IO · Prisma · PostgreSQL

Client: Flutter · Riverpod — Hosting: Render · Neon Postgres

Metric	Value
Backend code	10,884 lines of TypeScript across 72 files
Client code	21,637 lines of Dart across 92 files
Database	12 Prisma models, 5 migrations, PostgreSQL
Real-time API	60 Socket.IO events
REST API	~35 endpoints across 9 route modules
Commits	100

This document explains what the project is, why each technical choice was made, and how the harder features actually work. It is written to be read before an interview: every claim maps to code you can open.

1. What the project is

Nuno is a real-time multiplayer card game — UNO-style rules — playable on Android. Players sign in with Google, then either join a matchmaking queue to be paired with strangers, or create a private room and invite friends with a share code. A match runs live over WebSockets with a 20-second turn timer, voice chat, emotes and text chat.

Around the game sits the progression layer: XP and levels, an Elo-style rating with rank tiers, a global and friends leaderboard, a coin economy, and a cosmetics shop of 61 items (avatars, card backs, table themes, avatar frames, badges, titles, emotes).

The one design decision everything else follows from

The server is authoritative. The client never decides anything that matters. It does not know other players' cards, it does not decide whose turn it is, and it cannot declare itself the winner. It sends intent ("play this card id") and renders whatever state the server sends back.

This matters because the alternative — trusting the client — is how multiplayer games get cheated. The rule engine, the deck, the turn order and the win check all live on the server. A modified client can send anything it likes; the server rejects what is not legal.

```
// The client sends only an id. It does not send the card, its
// colour, or what it thinks should happen.
socket.emit('card.play', { cardId: 'a3f9...' });

// The server resolves the card from the player's own hand, checks
// ownership, checks it is their turn, checks the play is legal,
// then applies the effect and broadcasts new state to everyone.
```

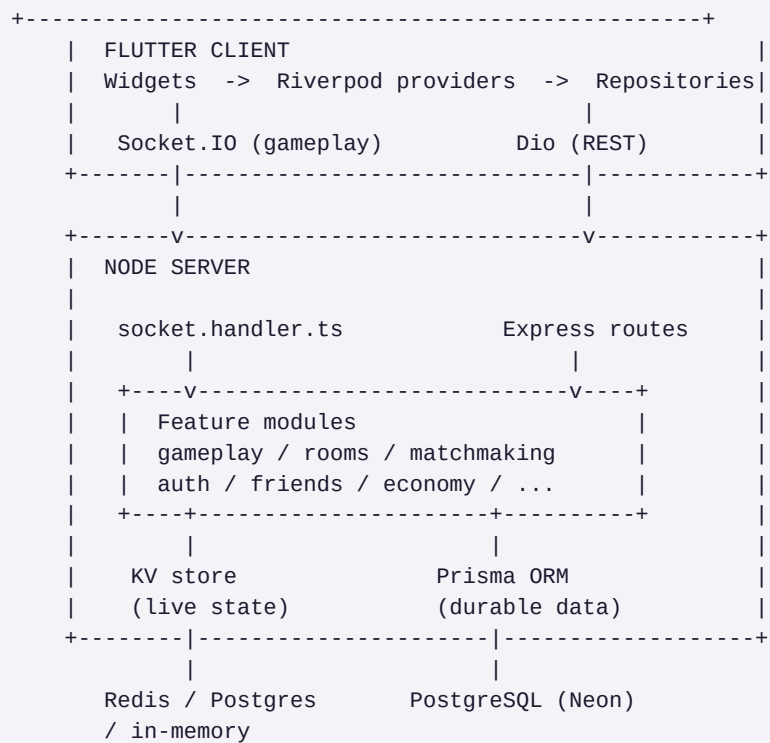
2. Technology stack, and why

Layer	Choice	Why this one
Language (server)	TypeScript	Compile-time safety on a domain full of enums — card colours, values, match states. A typo in a state name becomes a build error, not a runtime bug.

HTTP	Express 5	Only used for auth, profile, shop and leaderboard. Gameplay is not REST — it is push-based, so it belongs on a socket.
Real-time	Socket.IO 4	Rooms, automatic reconnection and a fallback transport out of the box. Raw WebSockets would have meant writing all three.
Database	PostgreSQL via Prisma 5	Relational data — users, friends, matches, cosmetics — with real foreign keys. Prisma gives typed queries and versioned migrations.
Auth	Google OAuth + JWT	No password to store, reset or leak. Short-lived access token plus a long-lived refresh token.
Validation	Zod	Request bodies are parsed into known shapes at the boundary, so no handler has to guess what it received.
Client	Flutter + Riverpod	One codebase, native performance. Riverpod because game state has to be shared across many widgets and survive navigation.
Voice	WebRTC (flutter_webrtc)	Peer-to-peer audio. The server only relays signalling, so voice does not consume server bandwidth.
Hosting	Render + Neon	Free tier, managed Postgres, deploy on git push.

3. Architecture

The three layers



Why the server splits storage in two

Live match state and durable records have completely different needs, so they go to different places.

	Live state (KV store)	Durable data (Postgres)
Holds	Current match, hands, whose turn, room membership, matchmaking queue	Users, friendships, finished matches, cosmetics owned, ratings
Lifetime	Minutes — deleted when the match ends	Forever
Access pattern	Read and write on every single card play	Written once when a match finishes
If lost	One match is disrupted	Unacceptable

Putting live match state in Postgres would mean a database write on every card. Putting user accounts in a KV store would mean losing them.

The KV store is an interface, not a product

src/config/redis.ts defines a Store interface and three implementations that satisfy it:

- RedisStore — used when REDIS_URL is set.
- PgStore — a key-value table inside Postgres. The production default, because the project runs on one instance and Redis free tiers have command quotas that a game blows through.
- InMemoryStore — a plain Map, for local development with no infrastructure at all.

Nothing else in the codebase knows which one is active. This was a deliberate design choice, and it paid off: switching from Redis to Postgres in production was a config change, not a rewrite.

4. The gameplay engine — the core of the project

`src/gameplay/` is the largest module at 2,256 lines. It is split so each file has one job:

File	Responsibility
<code>deck.engine.ts</code>	Builds the 108-card deck, shuffles it, deals hands, and rebuilds the draw pile from the discard pile when it runs out.
<code>rule.engine.ts</code>	Pure functions: is this play legal, whose turn is next, does this card match. No state is written here.
<code>game.engine.ts</code>	Applies a play: removes the card, resolves its effect, advances the turn, checks for a win, persists the new state.
<code>game.state.ts</code>	Reads and writes match state, and builds the per-player view.
<code>game.handler.ts</code>	Socket event handlers plus the turn timer.
<code>house.rules.ts</code>	The optional rule variants and their defaults.

4.1 Deck integrity

A real UNO deck is 108 cards and the distribution is not uniform — one zero per colour, two of every other number, two of each action card, four Wilds and four Wild Draw Fours. Getting this wrong is invisible until someone notices the odds feel off.

```
4 colours x 1 zero           = 4
4 colours x 9 numbers x 2    = 72
4 colours x 3 actions x 2    = 24
Wild                         = 4
Wild Draw Four               = 4
---
108
```

Every card gets a UUID at deal time. That is what makes "play this card" safe to express as an id: two red sevens are different objects, and the server can verify the specific card is in that player's hand.

4.2 The per-player view — how hands stay secret

The server holds one complete match state containing every hand. It never sends that object to anyone. `getPlayerState()` builds a different view per player:

```
myHand:      [full card objects]    // only yours
playerCardCounts: { p2: 5, p3: 3 }  // others: a number only
topCard, currentColor, currentTurn, ... // shared, public
```

Opponent hands leave the server as counts. There is nothing to intercept, because the information was never transmitted.

4.3 The turn timer, and the performance problem it caused

Each match has a 20-second turn limit, plus an auto-draw at 3 seconds if the player has nothing playable. The first implementation checked this once a second:

```
setInterval(async () => {
  const state = await store.get(`game:${matchId}`); // every second
  ...
}, 1000);
```

That is one store read per match per second, **whether or not anything is happening**. The cost scales with the number of open tables, not with player activity. 500 idle matches cost 500 reads/second.

The fix: a turn is only actionable at two known moments. After each check the timer computes when it next needs to look and skips every tick until then. A skipped tick is an integer comparison in process memory — no network call.

	Before	After
Reads per 20-second turn	20	2
500 concurrent matches	500 reads/sec	50 reads/sec

A related fix: broadcasting table state resolved usernames with a database query per player, per broadcast. A 5-minute TTL cache took a 4-player broadcast from 4 queries to 1, then 0.

5. House rules — configurable gameplay

Real UNO players almost never play the printed rules. The most common variant, stacking a +2 on a +2, is not official — Mattel has said so explicitly — but roughly half of all players use it.

So the game ships with the **official Mattel ruleset as the default**, and six opt-in variants the room host can tick in any combination:

Variant	Effect
Stack +2	Answer a +2 with your own +2; the penalty accumulates.
Stack +4	The same for +4. Cannot be mixed with +2 chains.
Jump-in	Play an identical card (same colour and number) out of turn. Numbers only — action cards would make turn order unresolvable.
Seven-Zero	A 7 swaps hands with a chosen player; a 0 rotates every hand one seat.
Draw to match	Keep drawing until a playable card appears.
Forced UNO	Miss the UNO call and draw two automatically.

Three decisions worth defending

Default is official. A rule set where variants are opt-in means a new player gets the game they expect. Every flag defaults to false, and a malformed request degrades to the official game rather than being rejected.

Variants only exist in private rooms. Quick Match pairs strangers by rating who have agreed to nothing, so a queue whose rules varied would be unplayable. The matchmaking path passes `OFFICIAL_RULES` explicitly rather than relying on the parameter default, so the guarantee is visible in the code and not an accident.

Stacking is same-type only. Research into how people actually play showed that groups who allow stacking still reject mixing +2 and +4 chains. The pending stack records its own card type and rejects the other.

```
// A stack is pending: the only legal answer is the same draw card.
if (state.pendingDraw && state.pendingDraw > 0) {
    return card.value === state.pendingDrawType;
}
```

The chain is also capped, so a long stack cannot empty the draw pile and deadlock the match.

6. Two ways into a game

6.1 Quick Match — the rating queue

The player picks a table size (2, 3, 4, 6 or 8) and joins a queue. The queue key is per mode **and** per size:

```
queue:{mode}:{tableSize}      e.g. queue:CASUAL:4
```

Splitting the key this way means a player who asked for a 2-player game can never be pulled into a 6-player table. When a queue reaches its required size, the server creates a room, deals a match and pushes `game.started` to everyone.

There is no background sweeper — the queue is processed on join. That is cheap and correct, but it created a bug worth describing (section 9.3).

6.2 Private rooms

A room gets a short share code. Players join by typing the code or by accepting a socket invite from a friend. The host controls the table size, the house rules, kicking, and the start; guests mark themselves ready. Starting runs a countdown that cancels if someone leaves or un-readies.

7. Progression, economy and social

7.1 XP and levels

The level curve is **generated, not hand-written**, so it cannot drift out of order:

```
step = 200 + (level - 1) * 50
XP_FOR_LEVEL[level] = XP_FOR_LEVEL[level - 1] + step
```

50 levels, quadratic — early levels arrive fast, later ones take real play. The same table is used when a match ends and when a profile is read, so the server and the progress bar can never disagree. Levelling up pays coins, with larger milestones at every 5th, 10th and 25th level.

7.2 Rating and tiers

Winning a match is +25 rating, losing is -15. Rating maps to seven tiers (Bronze through Grandmaster), each split into three divisions.

The tier is **derived from the rating on every read** rather than trusted from the stored column. That choice fixed a real bug: the original code incremented `rating` but never recomputed `tier`, so a 1020-rated player was still labelled "Bronze III" when Gold starts at 1000. Deriving on read meant every existing row became correct the moment the code shipped — no backfill migration needed.

7.3 Economy and cosmetics

61 catalogue items across seven cosmetic types. Coins come from playing matches, levelling up, and a 7-day daily reward streak. Purchases are server-priced (section 9.1). Equipping is exclusive per type, done in a transaction so a player can never end up with two card backs or none.

The daily reward uses an IST day boundary and an atomic increment as a claim guard, because the naive version could be claimed twice by tapping quickly.

7.4 Friends and presence

Friends are added by a public 10-digit player ID — the same idea as Free Fire or BGMI. The internal primary key is a UUID and is never shown; nobody is going to read 36 hex characters out to a friend.

Presence is resolved in strict priority order so the answer is never ambiguous:

```
OFFLINE > IN_MATCH > IN_LOBBY > ONLINE
```

Direct messages persist for 24 hours with an hourly sweep, so a message sent to an offline friend is still there when they log in.

8. The Flutter client

21,637 lines of Dart, organised by feature rather than by type — `features/game/`, `features/lobby/`, `features/store/` — so everything belonging to one screen sits together.

State management

Riverpod, because game state has to be shared across many widgets, survive navigation, and update from socket events arriving outside any widget's lifecycle.

Provider type	Used for
Provider	Dependency injection — repositories, the socket service
StateNotifierProvider	Complex mutable state — the game controller, auth, matchmaking
FutureProvider	Server-backed reads — inventory, leaderboard, rank

Keeping the socket and the REST session in sync

The two halves need each other: a refreshed access token has to reach the socket, and a handshake the server rejects has to trigger a refresh. Wiring that inside either provider would make them mutually dependent, and Riverpod cannot infer a type through a cycle. The callbacks live in a third provider so both stay standalone — a small piece of design that avoided a real architectural problem.

9. Bugs worth talking about

These are the most likely interview questions. Each one has a commit.

9.1 The shop could be robbed

`purchaseItem` trusted the price in the request body:

```
// The client said what the item cost.  
data: { coins: { decrement: input.price } }
```

Two exploits followed:

- `price: 0` bought any item free — including the 4,000-coin ones.
- `price: -100000` ran `decrement: -100000`. **Decrementing by a negative number adds.** The wallet went from 100 to 100,100.

The second is the more interesting one, because it is not obvious from reading the line. The fix: the request now carries only an item id, and price, type and availability are all resolved from the server-side catalogue.

9.2 One rejected promise could kill the server

Two calls in the post-authenticate block were floating promises — started, never awaited, no `.catch`. Node terminates the process on an unhandled rejection, so **one storage blip during one player's login would disconnect everyone in every match.**

Both are now caught. The server also gained process-level handlers: `unhandledRejection` logs and keeps serving, `uncaughtException` logs, closes the HTTP server and exits cleanly.

9.3 Ghost players in the matchmaking queue

Disconnecting released the player's room but not their queue entry, which had a one-hour TTL. Because the queue is only processed on join, the next player to arrive was paired with a socket that no longer existed — and waited in a match that would never start.

Fixed by releasing the queue entry in the disconnect path, alongside the room.

9.4 A sanitiser that quietly corrupted every array

An input-sanitising middleware rebuilt objects with a plain `{}` literal. In JavaScript an array **is** an object, so:

```
{ "uids": ["123", "456"] }      // what was sent  
{ "uids": { "0": "123", "1": "456" } }  // what the route saw
```

Every `Array.isArray()` check downstream failed, turning valid requests into 400s. This ran on every request body in the application, so it was never really a bug in one endpoint.

The lesson was in the test, not the fix. The endpoint's tests passed because they mounted the router alone, without the middleware stack the real server runs. A test that does not reproduce the real request path will happily pass against broken code.

9.5 A circular dependency in the client

Currency and rank were read by five screens through four independent caches, none of which expired, and the tabs are an `IndexedStack` so all five stay mounted. Whichever loaded first kept its value — the home header said 750 coins while the shop said 540.

The first fix made it worse. A central helper invalidated every cache from the auth controller, which created a cycle:

```
authController --invalidate--> inventoryProvider  
inventoryProvider --watch--> currentUserId
```

```
currentUserId    --watch-->    authController
```

`ref.invalidate` looks like a one-way command, but Riverpod records it as a dependency edge. The result was a `CircularDependencyError` on every button that refreshed the profile — buying, claiming, finishing a match.

The working fix inverts the arrows: a revision counter that depends on nothing, which each cache watches for itself. Bumping it re-runs them all, and the graph stays acyclic.

9.6 Cosmetics that equipped correctly and looked broken

Four avatar frames rendered as solid white squares over the player's portrait. The server was correct, the database was correct, the UI was correct — the PNGs had **no alpha channel**. They had been exported with the transparency checkerboard baked into the pixels.

Worth remembering as an example of a bug that is not in the code at all. The fix was a flood fill from the border and the hollow centre, plus a check that now fails the build if any overlay asset ships without transparency.

10. Security

Concern	How it is handled
Authentication	Google OAuth — the ID token is verified server-side against Google's public keys. No password is ever stored.
Sessions	Short-lived JWT access token, long-lived refresh token. The client re-handshakes the socket when the token is renewed.
Socket authorisation	Every handler checks <code>socket.userId</code> . An unauthenticated socket can reach nothing.
Server-authoritative play	Card legality, turn order, ownership and win conditions are all decided server-side.
Economy	Prices come from the catalogue, never the request.
Input	Zod schemas at the boundary, plus XSS stripping and prototype-pollution key filtering.
Rate limiting	<code>express-rate-limit</code> , with a tighter limit on auth.
Headers	Helmet, and a CORS allowlist.
Admin endpoints	Disabled entirely unless <code>ADMIN_TOKEN</code> is set, refuse a token shorter than 24 characters, and compare in constant time.

11. Limitations, and what I would change

Being able to answer this honestly is worth more than pretending the project is finished.

- **Single instance.** Socket.IO only attaches its Redis adapter when `REDIS_URL` is set. Scaling to a second instance needs Redis, and I measured what that would cost before deciding to defer it — a complete 4-player game is roughly 268 store commands, which puts ~50 games/day at 80% of a typical free tier. That measurement is why the project runs on Postgres instead.
- **No automated CI.** Tests and checks are run manually. They should run on every push.
- **Rating is not true Elo.** Flat +25/-15 ignores opponent strength. Real Elo would be a better model.
- **Some UI is ahead of the backend.** Achievements and a season pass are designed but not wired to real data.
- **Reconnection is partial.** Match state is restored, but a player who disconnects mid-turn loses that turn to the timer.

12. Likely interview questions

Q. Why Socket.IO instead of plain WebSockets?

Three things I would otherwise have written myself: rooms (broadcasting to four players without tracking sockets manually), automatic reconnection with backoff, and a polling fallback for restrictive networks. On mobile, connections drop constantly — reconnection alone justified it.

Q. How do you stop a modified client from cheating?

The client is never trusted with anything that matters. It does not receive other players' cards, so it cannot leak them. It sends a card id, and the server verifies the card is in that player's hand, that it is their turn, and that the play is legal. Prices come from the catalogue, not the request — that one was a real exploit I found and fixed.

Q. Why Postgres for live match state instead of Redis?

I measured it first. One live 4-player table is about 13.7 KB, so memory was never the constraint — command volume was. A full game is roughly 268 store commands, and the free tiers I looked at cap commands, not memory. Since the app runs on a single instance, Postgres handles the load comfortably. The store is behind an interface, so moving to Redis is a config change when a second instance is needed.

Q. What was the hardest bug?

A circular dependency I introduced while fixing something else. Currency was stale across screens, so I added a central invalidator — which made the auth controller depend on the providers that already depended on it. `ref.invalidate` reads like a command but Riverpod treats it as a dependency edge, so every button that refreshed the profile threw. The fix was to invert the direction: a revision counter that depends on nothing and that each cache watches for itself.

Q. How did you find the performance problem?

I counted store operations across a full match rather than guessing. That showed the turn timer was reading state once per second per match — cost scaling with open tables rather than with player activity. Since a turn is only actionable at two moments, the timer now computes when it next needs to look and skips every tick until then. 20 reads per turn became 2.

Q. Why are house rules only available in private rooms?

Consent. In a private room everyone can see the rules before the game starts. In Quick Match you are paired with a stranger by rating and have agreed to nothing, so varying the rules there would be unplayable. The matchmaking path passes the official rule set explicitly rather than relying on a default, so the guarantee is visible in the code.

Q. What would you do differently?

Set up CI from day one — every test I wrote is run by hand, which is a discipline that does not survive a deadline. I would also design the shared-state layer for multiple instances from the start rather than retrofitting the adapter.

Every claim in this document maps to code in the repository. The strongest answer to "can you prove it" is to open the commit.